CHPMS

Snapshot-Mode Failover Architecture

Architectural & Implementation Plan

Project: Coconut Husk Processing Management System

Document version: 1.0

Date: 2026-05-10

Status: Awaiting stakeholder decisions

Audience: Project owner, infrastructure operator, development team

Table of Contents

1. Problem statement & objectives

1.1 Current state

CHPMS runs as a single Docker stack on a designated **host machine**: PostgreSQL + Next.js application + nightly backup container. Other team members reach the system over **Tailscale**, browsing to the host's Tailscale hostname. All traffic terminates at the host's Next.js process; all data lives in the host's PostgreSQL instance.

1.2 The failure mode

When the host machine loses power, network, or is otherwise offline, every client loses access to the system entirely. There is currently no mechanism for clients to view recent data, no off-host backup, and no clear indication to users that the system is in a degraded state — they simply see a connection error.

1.3 Objectives of this plan

#	Objective
O1	Each client machine should automatically see **live data** when the host is online.
O2	When the host is offline, clients should automatically see the **most recent snapshot** from Google Drive.
O3	Mode transitions must be **automatic** — no tab switching, no separate URLs, no user intervention.
O4	Snapshot mode is **strictly read-only**; all write actions must be blocked at both UI and API layers.
O5	The user must be clearly informed: *which* mode is active, *when* the snapshot was taken, and *that the live system is unreachable*.
O6	Snapshots are persisted to a **Google Drive shared folder** on a 15-minute cadence.
O7	The implementation must reuse existing scaffolding (`SYSTEM_MODE` , `SNAPSHOT_TIMESTAMP` env vars, the existing `backup` service).

2. Architecture overview

2.1 Core idea

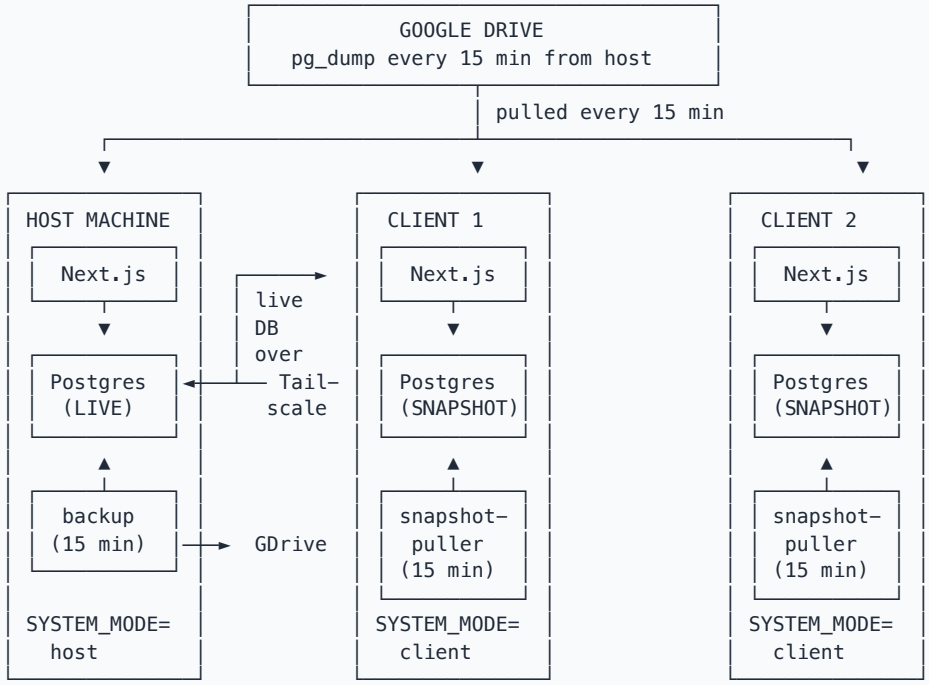
Every machine — host and clients — runs the **same** Docker stack. The difference is at the database connection layer: each client's Next.js instance dynamically chooses which database to read from based on whether the host is reachable.

- **Host reachable** → query the host's *live* PostgreSQL over Tailscale → app shows **LIVE** mode

- **Host unreachable** → query the *local snapshot* PostgreSQL → app shows **SNAPSHOT** mode (read-only)

The user always opens `http://localhost:3000` (or their machine's Tailscale name). The mode switch is invisible; the same URL always works.

2.2 Topology



2.3 How this satisfies each objective

#	Objective	Mechanism
O1	Live when host is up	Per-request router selects `livePrisma`; queries hit host's DB over Tailscale; full read/write
O2	Snapshot when host is down	Heartbeat fails, router flips to `snapshotPrisma`; reads served from local snapshot DB
O3	Automatic, single URL	Switching is server-side, per request; user never changes URLs
O4	Read-only in snapshot	Middleware returns 503 on writes; UI buttons disabled
O5	Clear indication	Persistent banner with mode + snapshot timestamp + reason
O6	15-minute GDrive cadence	Modified `backup` container with `rclone` + cron `*/15 * * * *`
O7	Reuse scaffolding	`SYSTEM_MODE` already wired; extend semantics to `host` | `client`

3. Mode-switching mechanism (runtime detail)

Inside each client's Next.js process, four pieces work together to deliver a seamless live ↔ snapshot transition.

3.1 Two PrismaClient instances at startup

```
// src/lib/prisma-router.ts (new file)
const livePrisma = new PrismaClient({
  datasources: { db: { url: process.env.LIVE_DATABASE_URL } },
});
const snapshotPrisma = new PrismaClient({
  datasources: { db: { url: process.env.SNAPSHOT_DATABASE_URL } },
});
```

3.2 Background heartbeat (every 10 seconds)

A self-scheduling timer in the Node.js process attempts a 2-second `SELECT 1` against `livePrisma`. The result is cached in process memory along with the last-success timestamp. Three consecutive failures (~30 s) flip the cached state to *offline*.

3.3 Per-request router

```
// Returns the right Prisma client for the current request.
export async function getPrisma(): Promise<{
  client: PrismaClient;
  mode: "live" | "snapshot";
  snapshotAt: string | null;
}> {
  if (heartbeat.isLive()) return { client: livePrisma, mode: "live", snapshotAt: null };
  return {
    client: snapshotPrisma,
    mode: "snapshot",
    snapshotAt: readSnapshotMeta()?.takenAt ?? null,
  };
}
```

3.4 Write guard middleware

When mode is `snapshot`, all `POST`, `PUT`, `PATCH`, and `DELETE` requests receive HTTP `503` with a JSON body explaining the snapshot mode and timestamp. The UI never sends these requests in snapshot mode (buttons are disabled), but the middleware is a defense-in-depth backstop.

3.5 UI banner

A sticky top banner subscribes to the mode (via a small client polling endpoint every ~5 s). Two states:

- **LIVE**: small green dot, no banner — system feels normal.
- **SNAPSHOT**: amber sticky banner —
  "📸 *System unreachable. Showing snapshot from {timestamp}. Read-only mode.*"

4. Implementation plan (phased)

Phase 1 — Expose host's PostgreSQL over Tailscale (host change only)

File	Change
`docker-compose.yml`	Bind PostgreSQL port to Tailscale interface (or 0.0.0.0 with Tailscale ACL restriction)
`postgres/pg_hba.conf` *(new)*	Restrict connections to the Tailscale subnet (`100.64.0.0/10`) with `md5` authentication
`.env.example`	Document `TAILSCALE_NET` and `DB_HOST_FOR_CLIENTS`

Phase 2 — Backup pipeline (host change only)

File	Change
`backup/Dockerfile` *(new)*	Base on `postgres:16-alpine` + install `rclone`
`backup/backup.sh`	After `pg_dump`, upload to GDrive via `rclone copy`; prune by retention policy
`docker-compose.yml`	Cron `*/15 * * * *`; mount GDrive service-account JSON; add `GDRIVE_FOLDER_ID` env
`.env.example`	New vars: `GDRIVE_SERVICE_ACCOUNT_JSON_PATH`, `GDRIVE_FOLDER_ID`

Retention policy: 96 fifteen-minute snapshots (24 hours) plus 7 daily snapshots, pruned by filename pattern.

Phase 3 — Snapshot puller (client side, new container)

File	Change
`snapshot-puller/Dockerfile` *(new)*	`postgres:16-alpine` + `rclone`
`snapshot-puller/pull-and-restore.sh` *(new)*	Atomic restore: download → restore into `chpms_staging` → swap names. Writes `/app/data/snapshot-meta.json` with timestamp.
`docker-compose.client.yml` *(new)*	Overlay file that activates client mode: replaces `backup` with `snapshot-puller`, sets `SYSTEM_MODE=client`, points app at the local restored DB and at the host's live DB

Atomic restore strategy: restore into a side database, then atomically rename — total transition ~200 ms, no app downtime during snapshot refresh.

Phase 4 — Mode-switching Prisma router (app code, runs in all instances)

File	Change
`src/lib/prisma-router.ts` *(new)*	Holds `livePrisma` and `snapshotPrisma`; runs background heartbeat; exports `getPrisma()` returning `{ client, mode, snapshotAt }`
`src/lib/prisma.ts`	Replace direct export with a wrapper that delegates to `getPrisma()`
`src/lib/queries/*.ts`	Mechanical change: `prisma.x.findMany(…)` → `(await getPrisma()).client.x.findMany(…)`
All API route handlers	Same mechanical change
`src/middleware.ts` *(or extension)*	Block `POST/PUT/PATCH/DELETE` when mode is snapshot; return 503 with `{ error, mode, snapshotAt }`
`src/lib/system-mode.ts` *(new)*	Server helpers: `requireLive()`, `getSystemModeForUI()`

Phase 5 — UI banner + write button gating

File	Change
`src/components/shared/system-mode-banner.tsx` *(new)*	Sticky top banner. Polls `/api/system/mode` every 5 s. Renders LIVE/SNAPSHOT differently.
`src/app/api/system/mode/route.ts` *(new)*	Returns `{ mode, snapshotAt, lastLivePingAt }` (no auth required)
`src/app/(authenticated)/layout.tsx`	Mount the banner above the page chrome

`src/lib/use-system-mode.ts` *(new)*	Client React hook exposing mode to action components
Action components (FulfillForm, RecordPayment dialogs, status change buttons, edit/delete actions, "Add new" buttons)	Disable when `mode === "snapshot"` with tooltip "*System is in snapshot mode (read-only)*"

Strategy: introduce a single `<ActionGuard>` wrapper that disables children in snapshot mode. Wrap once at the layout/page level rather than touching every button site individually.

Phase 6 — Documentation + bootstrap

File	Change
`SETUP.md`	New section: *Setting up a client machine for snapshot fallback* — Tailscale install, GDrive credentials, command to bring stack up
`scripts/setup-client-machine.sh` *(new)*	Interactive wizard: prompts for primary's Tailscale name, GDrive folder ID, drops `.env.client`, runs `docker compose up`
`README.md`	Architecture diagram + brief mode explanation at the top

5. Information required from stakeholders

Before implementation can begin, the following information and decisions are required. Each is grouped by the party best positioned to provide it.

5.1 From the project owner / business

#	Question	Default if no answer
Q1	Confirm that snapshot mode must be **strictly read-only** (no offline writes, no eventual sync back to the live system).	Yes — strictly read-only.
Q2	How long is acceptable as the maximum data-loss window if the host fails between backups?	15 minutes.
Q3	How many client machines need snapshot capability? (Affects setup overhead, GDrive bandwidth.)	To be determined.
Q4	Should snapshot mode require a logged-in session, or can the snapshot be browsed without authentication on the LAN?	Logged-in session required (uses snapshot's user table).

5.2 From the infrastructure operator / IT

#	Question	Required artifact

Q5	Tailscale ACL configuration: confirm the host's PostgreSQL port (5432) can be exposed to authorized client devices on the tailnet, and supply the ACL excerpt or confirm an existing ACL covers it.	Tailscale ACL JSON or written confirmation.
Q6	Google Drive: provide a **Google Cloud service account** JSON key with permission to upload to and download from a designated shared folder. (Service account is preferred over user OAuth for unattended/headless operation.)	Service account JSON file + the GDrive folder ID it has access to.
Q7	Designated host machine: confirm hostname, Tailscale name, expected uptime, who is responsible for restarting it, and where backups of the host's local DB should ALSO be stored (in addition to GDrive) for disaster recovery.	Document.
Q8	Confirm each client machine has Docker installed and at least 2 GB free disk for the local snapshot database.	Inventory.

5.3 From the development team (configuration choices)

#	Question	Recommendation
Q9	Heartbeat interval and stale threshold (live → snapshot flip)?	10 s poll, 30 s threshold (3 misses).
Q10	Snapshot restore strategy?	Atomic swap (no app downtime during refresh).
Q11	Banner UX: passive top stripe vs modal-style block-until-acknowledged?	Passive stripe + disabled action buttons.
Q12	Write-attempt UX in snapshot mode: hard-block (button disabled) vs allow click + show error toast?	Hard-block at the UI; 503 from the API as defense-in-depth.

6. Effort, risk & rollout sequence

Phase	Effort	Risk	Independent?
Phase 1 — Postgres on Tailscale	S	**MED** (security review)	Yes
Phase 2 — GDrive backup	M	**LOW**	Needs rclone setup; can test independently
Phase 3 — Snapshot puller	M	**LOW**	Needs Phase 2
Phase 4 — Prisma router	L	**HIGH** — touches every query & route	Yes (can stub heartbeat to always-live until Phase 3 ready)

Phase 5 — UI banner + gating	M	**LOW**	Needs Phase 4
Phase 6 — Docs + bootstrap	S	—	Last

Total estimated effort: approximately 2–3 days of focused development. Phase 4 dominates because the database access pattern changes from a global singleton to a request-scoped router, requiring careful and systematic edits across all query and API files.

Recommended rollout order

1. Set up the Google Cloud service account and Drive folder (out-of-band).
2. Implement Phase 1 (Tailscale exposure) and verify host's DB is reachable from a test client.
3. Implement Phase 2 (backup pipeline) and verify snapshots appear in GDrive on the 15-minute cadence.
4. Implement Phase 3 (snapshot puller) on one client machine and verify automatic restore.
5. Implement Phase 4 (Prisma router) — staged: ship with heartbeat returning `always-live` first, verify nothing regresses; then enable real heartbeat.
6. Implement Phase 5 (banner + gating).
7. Roll out to remaining client machines via Phase 6 bootstrap.

7. Appendix — file change inventory

New files

- `backup/Dockerfile` — backup image with `rclone`
- `postgres/pg_hba.conf` — Tailscale-restricted auth rules
- `snapshot-puller/Dockerfile` — puller image
- `snapshot-puller/pull-and-restore.sh` — atomic restore script
- `docker-compose.client.yml` — overlay for client machines
- `scripts/setup-client-machine.sh` — interactive bootstrap wizard
- `src/lib/prisma-router.ts` — dual-client router with heartbeat
- `src/lib/system-mode.ts` — server-side mode helpers
- `src/lib/use-system-mode.ts` — client-side mode hook
- `src/components/shared/system-mode-banner.tsx` — UI banner
- `src/components/shared/action-guard.tsx` — wrapper that disables children in snapshot mode
- `src/app/api/system/mode/route.ts` — mode status endpoint

Modified files

- `docker-compose.yml` — Postgres binding, backup cadence, GDrive volume mount
- `backup/backup.sh` — GDrive upload step + retention pruning
- `.env.example` — new vars documented
- `SETUP.md` — client machine setup section added
- `README.md` — architecture diagram + mode summary
- `src/lib/prisma.ts` — delegates to `prisma-router`
- `src/middleware.ts` — extend with snapshot-mode write guard
- `src/app/(authenticated)/layout.tsx` — mount banner
- **All** `src/lib/queries/*.ts` — switch to `(await getPrisma()).client`
- **All** `src/app/api/**/*.ts` — same

Document control

Field	Value
Document	CHPMS — Snapshot-Mode Failover Architecture
Version	1.0
Date	2026-05-10
Status	Draft — awaiting decisions on Q1–Q12 in Section 5
Owner	CHPMS development team